

Retrieval Quality

Design Notes - Updated with Encoder Benchmark

Working notes on what makes a retrieved chunk good, how to measure it, and the current controlled embedding-model benchmark.

Core principle

The only meaningful ground truth is whether retrieved evidence enables the LLM to answer correctly. Retrieval similarity, reranker scores, and structural heuristics are proxies for that outcome.

0. First action - build the harness

Before implementing the hierarchical index, build the test harness. It works against FAISS baselines immediately, so it is testable on day one.

1. A small labelled set - 50 queries, known correct chunk
2. Ground truth - FAISS Flat, exact
3. One function - run(retriever) -> recall@k, latency, computations
4. One results table - one row per configuration

Every open question below becomes a row in that table.

1. Incremental Indexing

Chosen strategy: nearest centroid + nightly rebuild.

```
def add_document(new_vector, doc_id):
    leaf = search(new_vector, max_backtracks=0)
    leaf_vectors[leaf].append((doc_id, new_vector))

# overnight
def reindex():
    new_index = build_hierarchical_index(load_all_vectors())
    atomic_swap(current_index, new_index)
```

Documents are immediately searchable. Index quality degrades slowly during the day and is restored by the nightly rebuild.

Monitoring: keep a per-leaf counter. If any leaf exceeds about 2x expected size before rebuild, flag it.

Honest limitation: HNSW supports native dynamic insertion. Tree structures do not.

2. What Makes a Hit a Hit

A chunk is a hit if it lets the LLM answer correctly. Everything else is an approximation.

Type	Example: capital of France?	Wanted?
Topically related	"France has a long history of culture"	No
Answer-bearing	"Paris is the capital of France."	Yes
Supporting evidence	Needed together with other chunks	Sometimes

Embeddings measure topical relatedness. We want answer-bearing evidence. That mismatch is the core problem.

Hierarchy of signals

Human says "yes, this answered it" -> ground truth, expensive, slow
LLM faithfulness / utilisation -> good, requires LLM call

Cross-encoder relevance -> better than cosine, one model call
Cosine similarity above threshold -> fast, cheap, noisy

3. Four Structural Failure Modes

Query echo: restates the question rather than answering it.

Entity substitution: correct structure, wrong subject.

Assertion vs mention: entities are present but the required claim is not asserted.

Answer-type mismatch: the chunk lacks the entity/date/quantity/explanation the question requires.

Common thread: embeddings encode aboutness. These failure modes can be about the right topic but still fail as evidence.

4. Three Cheap Detectors

Entity coverage

```
coverage = |entities(query) intersection entities(chunk)| / |entities(query)|
```

Answer-type match: classify expected answer type and check whether the chunk contains it.

Echo ratio

```
echo = token_overlap(query, chunk) / chunk_length
```

The weighted combination of these signals remains an experimental design choice rather than an established standard.

5. Filters Can Destroy Signal

This is the most important caveat in this document.

Entity coverage can fail on synonymy; answer-type matching can fail on explanation questions; echo detection can penalise legitimate FAQ or guideline content.

A filter's false negatives are invisible. A right chunk that was filtered is simply gone.

Safe pattern

```
retrieve 20 by cosine <- recall preserved  
reorder by combined signal <- precision improved  
send top 5 <- nothing deleted, only demoted
```

Run the harness with and without each filter. If recall@10 drops, the filter is destroying signal.

6. More Context Is Not Better

Long context can suffer from lost-in-the-middle effects and distractor sensitivity. Faster retrieval is better spent on more reranker candidates, more backtracking, or lower latency at fixed k - not automatically on more prompt tokens.

Testable experiment

```
k = 3, 5, 10, 20, 50 -> answer accuracy
```

7. Open - for later

Confidence estimation: score gap, top-k entropy, random-document-normalised score, backtrack count, and vector/BM25 agreement.

Click / feedback learning: later learning-to-rank layer using implicit feedback.

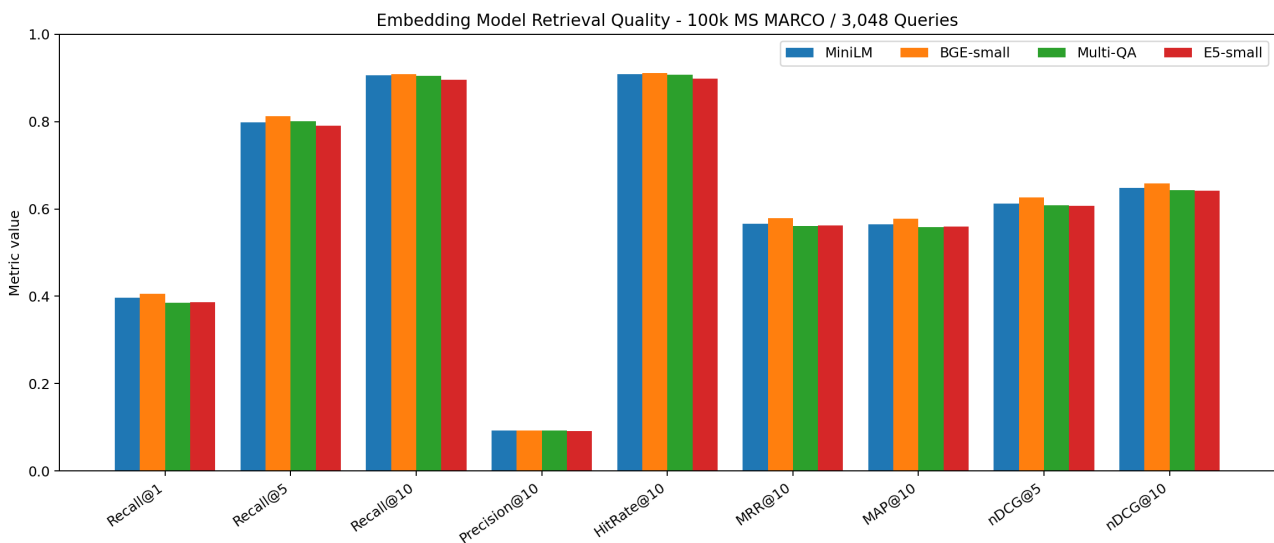
Adaptive k: vary chunk count by confidence rather than fixing it globally.

8. Controlled Embedding Model Benchmark

The following benchmark compares four 384-dimensional encoders under the same retrieval setup: **100,000 MS MARCO documents, 3,048 judged queries, FAISS Flat exact search, top-10 retrieval.**

This is the project's current apples-to-apples encoder comparison. Higher is better for every metric shown.

Metric	MiniLM	BGE-small	Multi-QA	E5-small
Recall@1	0.3962	0.4050	0.3853	0.3866
Recall@5	0.7988	0.8130	0.8012	0.7912
Recall@10	0.9057	0.9090	0.9053	0.8962
Precision@10	0.0927	0.0931	0.0927	0.0918
HitRate@10	0.9081	0.9108	0.9072	0.8980
MRR@10	0.5667	0.5792	0.5606	0.5620
MAP@10	0.5642	0.5775	0.5580	0.5597
nDCG@5	0.6122	0.6270	0.6085	0.6074
nDCG@10	0.6478	0.6589	0.6432	0.6422



Result: BGE-small wins all nine measured metrics. Its advantage is modest at Recall@10 and HitRate@10, but more visible in ranking-sensitive measures such as MRR, MAP, and nDCG.

Current encoder choice: BAAI/bge-small-en-v1.5. This is the current 384-dimensional baseline to carry forward into the ANN/index experiments.

Suggested experimental order

- Freeze BGE-small as the encoder baseline.
- Use the same vector set to compare FAISS Flat, HNSW, IVF, and the hierarchical ANN design.
- Measure recall@k against exact Flat ground truth together with latency, computations, memory, and build cost.
- Add reranking and structural retrieval-quality experiments only after the ANN baseline is stable.